

DEXOAK: Object-Aware Kinematic Retargeting

Abstract—We introduce DEXOAK, a learning-based object-aware kinematic retargeting framework for generating contact-consistent robot hand references from human motion-capture data. DEXOAK first minimizes an interaction-graph Laplacian objective over a joint hand–object graph, so the robot reproduces the demonstrated object-relative interaction geometry rather than the human hand pose alone. These per-frame solutions form seed retargeted references, from which a feed-forward network conditioned on human keypoints and object point clouds learns the retargeting function. Object-aware fine-tuning then refines the network under contact, non-penetration, and self-collision losses that position the fingers on the demonstrated contacts and reduce artifacts destabilizing physics-based refinement. Once trained, the network transfers to unseen data by inference alone, without per-frame optimization.

DEXOAK references improve dynamic retargeting over the state-of-the-art across bimanual tasks with rigid and articulated objects. On the Inspire hand, success rises from 39.5% to 48.4% on OakInk-v2, and ADD-AUC rises from 62.7% to 77.9% on ARCTIC. Ablations trace the gains to object-aware kinematic retargeting and the amortized network refined with contact losses. Trained on OakInk-v2, it transfers to ARCTIC zero-shot and nears in-domain performance after fine-tuning. On hardware, 46/56 open-loop replays succeed across five tasks, enabling dexterous robot data generation from human data.

I. INTRODUCTION

Progress in robot manipulation increasingly comes from learning at scale, yet the resulting policies [1, 2] remain bounded by scarce robot interaction data. It cannot be scraped from the web like the image and text corpora behind vision and language models, and teleoperation, the most reliable alternative, is slow and costly. The gap is widest for dexterous manipulation, where multi-fingered hands are hard to teleoperate [3], leaving contact-rich interaction especially scarce. Abundant clips of people handling objects, increasingly annotated with hand and object pose [4, 5, 6], are a natural source of the dexterous interaction data that robots lack. Such data supports robot learning through representation learning [7], policy pretraining [8], imitation learning [9], or retargeting [10]. We focus on retargeting because it directly converts human demonstrations into dynamically feasible robot trajectories [11, 12], bypassing teleoperation entirely.

A human-to-robot retargeting pipeline has two stages. Kinematic retargeting sets *where* the robot hand goes relative to the environment, and dynamic retargeting sets *how* it actuates to interact with the object. Kinematic retargeting most often matches the robot and human vectors between hand keypoints [13, 14], fitting hand shape alone and ignoring the object. Yet human and robot hands differ in degrees of freedom (DoF) and embodiment geometry, so shape similarity does not guarantee task-equivalent interaction. Instead, recent

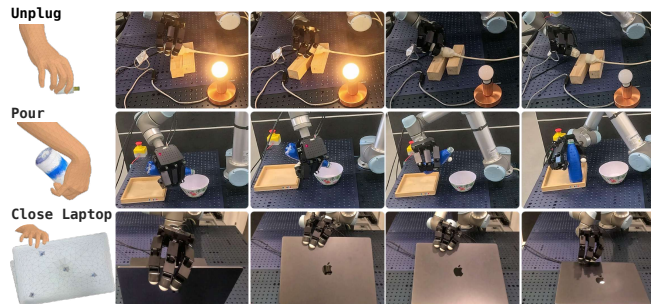


Fig. 1. DEXOAK enables dexterous real-robot data generation from human motion-capture. Human demonstrations of *Unplug*, *Pour*, and *Close Laptop* (held out from retargeter training) are successfully replayed open-loop on a UR5 + right Allegro using DEXOAK references refined by RL.

work [15] preserves robot–environment relations with per-frame interaction-graph optimization, but targets whole-body loco-manipulation rather than dexterous finger-level contact.

We argue that kinematic retargeting should preserve task function rather than hand-pose fidelity. What matters is the human–object interaction: object-relative fingertip placement, grasp approach, and object motion. Prior pipelines introduce object and contact information only during physics-based refinement [10, 11, 12], and never refine the kinematic reference itself under contact objectives. We therefore propose DEXOAK, which learns object-aware kinematic references before dynamic refinement. Every reference still requires physics refinement, but its quality determines how much error the dynamic stage must correct. We hypothesize that object-aware references let it focus on dynamics rather than inferring contact locations, improving performance.

DEXOAK is a two-stage pipeline that turns human hand–object motion-capture demonstrations into kinematic robot trajectories. An optimization-based retargeter builds paired human–object and robot–object interaction graphs [15] and solves for robot wrist pose and joint angles matching their Laplacian coordinates. Solving these per-frame non-linear programs sequentially yields object-aware kinematic references, our *seed retargeted references*. However, this optimization is sensitive to initialization [16] as contact and collision terms would make the objective more non-convex. We therefore train a differentiable network, sharing structure across tasks, to reproduce the seed references with one forward pass per frame instead of per-frame optimization. This supervised amortization initializes *object-aware fine-tuning*, whose surface-contact, object-part, and contact-direction losses improve robot finger placement beyond the optimizer outputs. Importantly, the fine-tuned network transfers to unseen data without a single per-frame solve. DEXOAK’s kinematic references act as the base action for a residual reinforcement learning (RL) stage [11, 17] that

corrects the remaining dynamics gap. Successful simulation rollouts are then replayed open-loop on hardware, demonstrating physical executability and yielding real-robot trajectory data from human motion-capture data inputs (Fig. 1).

This paper makes three contributions:

- (i) We present DEXOAK, a kinematic retargeting pipeline that preserves the hand–object interaction rather than hand pose alone, pairing an interaction-graph Laplacian optimizer with a feed-forward network that amortizes the seed retargeting and is refined under object-aware contact losses.
- (ii) We show that object awareness at the kinematic stage improves downstream RL, raising success from 20.6% to 48.2% over hand-only references. The learned retargeter transfers to an unseen dataset zero-shot or with light fine-tuning, without repeating optimization or amortization.
- (iii) We surpass state-of-the-art (SOTA) on OakInk-v2 [6] and ARCTIC [5] with Allegro [18] and Inspire [19] hands, and achieve 46/56 successful hardware replays across 5 tasks.

II. RELATED WORK

Learning manipulation from human data. Generalist robot policies [1, 2] draw action supervision from large, costly robot-trajectory datasets tied to particular hardware. Human data offers an abundant alternative and enters robot learning at two levels. At the representation level, it pretrains visual features, latent plans, or human–robot aligned embeddings [7, 9, 20], without ever mapping motion to the robot. At the action level, methods predict human actions that are retargeted to the robot [8], or learn policies and controllers from reconstructed videos and kinematic human–object references [21, 22, 23, 24], which first requires expressing the demonstration in the target embodiment. DEXOAK supplies that step, generating the object-aware references these pipelines consume.

Retargeting from human data. *Kinematic retargeting* maps human motion to robot configurations without physics. Hand-centric DexPilot [14] and AnyTeleop [13] optimize keypoint vectors, while GeoRT [25] learns a direct map from human finger keypoints to robot joints through geometric objectives. None constrains object-relative contact. OmniRetarget [15] instead preserves body–object–terrain relations through an interaction mesh and sequential constrained optimization, but targets humanoid whole-body loco-manipulation. *Dynamic retargeting* enforces physical executability. ManipTrans [11] pairs a hand-motion imitator with task-specific residual RL driven by object-tracking and contact rewards. DexMachina [12] learns object-state tracking with RL under a curriculum that progressively removes virtual object-control assistance, guided by explicit motion and contact rewards. SPIDER [10] instead uses sampling-based optimization, refining simulated control sequences through annealed sampling with virtual contact guidance. These recover physical plausibility, but receive an object-agnostic reference and must discover contacts themselves. DEXOAK instead places the object in the kinematic stage so the dynamic stage starts from a reference carrying the demonstrated contact geometry.

Concurrent pipelines [26, 27, 28] preserve hand–object structure with Laplacian objectives, but re-optimize each new frame or demonstration before RL. DEXOAK instead uses optimization only for training seed references. Its reusable retargeter is fine-tuned dataset-wide with object-aware losses to retarget new inputs without further optimization.

Hand–object interaction. DexYCB [4], ARCTIC [5], and OakInk-v2 [6] support contact-aware reconstruction and synthesis. ContactOpt [29] refines human pose with predicted contact. Grasp’D [30] synthesizes human and robot grasps through differentiable contact. ArtiGrasp [31] uses physics-based RL for bimanual grasping and articulation. Unlike these reconstruction and synthesis methods, DEXOAK uses contact attraction, contact direction, and penetration objectives for human-to-robot trajectory retargeting.

III. DEXOAK: OBJECT-AWARE KINEMATIC RETARGETING

DEXOAK maps a T -frame human hand–object motion-capture sequence to a kinematic robot trajectory (Fig. 2). At frame t , the inputs are N_h indexed human hand keypoints $K_t = (k_t^i)_{i=1}^{N_h} \in \mathbb{R}^{N_h \times 3}$ from SMPL-X [32], and an N_p -point object cloud $P_t \in \mathbb{R}^{N_p \times 3}$ sampled from the object meshes, both in the human wrist frame with pose $W_t \in \text{SE}(3)$. It outputs a robot wrist pose $w_t = (p_t, R_t) \in \text{SE}(3)$, with $p_t \in \mathbb{R}^3$ and $R_t \in \text{SO}(3)$, and J joint angles $q_t \in \mathcal{Q} = [q_{\text{low}}, q_{\text{high}}] \subset \mathbb{R}^J$. As w_t is wrist-relative, the capture-frame robot wrist is $W_t w_t$, making the target invariant to global placement. We produce one trajectory per hand, coupling bimanual tasks only in the dynamic stage (Sec. IV).

Kinematic retargeting proceeds in two steps. An interaction-graph Laplacian optimizer (left of Fig. 2) builds paired human–object and robot–object graphs, matches their Laplacian coordinates, and retargets each frame into a seed retargeted reference (Sec. III-A). A feed-forward network (right of Fig. 2) amortizes these solutions (Sec. III-B.1) and is then fine-tuned with object-aware contact losses (Sec. III-B.2). A separate stage (Sec. IV) then makes the trajectory physically executable.

A. Interaction-Graph Laplacian Optimization

Interaction Graph (IG). At frame t , we build two point sets. The source set $S_t = \{k_t^1, \dots, k_t^{M_h}\} \cup \{o_t^1, \dots, o_t^{M_o}\}$ holds M_h human hand keypoints (fingertips and wrist) from K_t , and M_o object anchors subsampled from P_t . The target set $\mathcal{T}_t(w_t, q_t) = \{r_t^1, \dots, r_t^{M_h}\} \cup \{o_t^1, \dots, o_t^{M_o}\}$ keeps the same object anchors and replaces the hand keypoints with robot counterparts. Equal hand-node indices in S_t and $\mathcal{T}_t$ denote anatomical matches, such as thumb to thumb. Thus, $r_t^i = w_t \text{FK}^i(q_t)$, where forward kinematics FK^i places robot point i in the robot root frame. Both sets have $N = M_h + M_o$ nodes. The shared object anchors make the Laplacian coordinates encode hand placement relative to the demonstrated object, unlike a hand-only graph (*IG (w/o object)*) vs. *IG (+ object)* in Fig. 3).

Following OmniRetarget [15], we apply a 3D Delaunay tetrahedralization to S_t . The neighborhood set $\mathcal{N}(i)$ contains

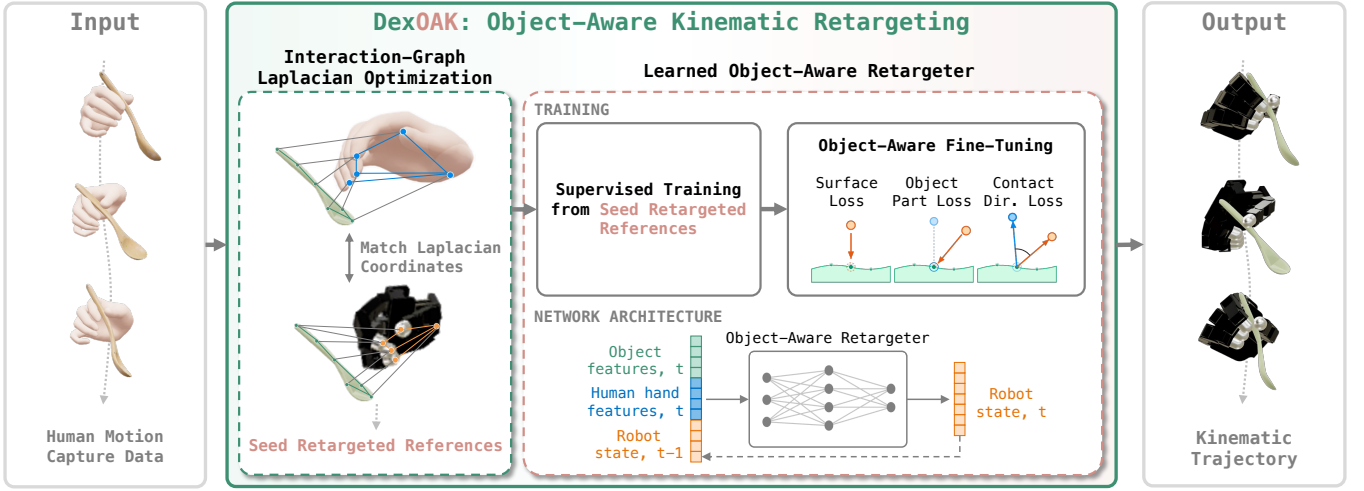


Fig. 2. **Overview of DEXOAK.** From a human hand–object demonstration, our interaction-graph Laplacian optimizer matches human and robot Laplacian coordinates to produce a per-frame seed retargeted reference. A learned retargeter amortizes this optimization, autoregressively mapping the object cloud and human keypoints to the robot wrist and joints. It is trained on the seed references and further refined with object-aware losses. The resulting kinematic trajectory is later refined by an established residual RL stage (Sec. IV, omitted from the figure). Colors: object (green), human (blue), robot (orange).

the nodes connected to i by a Delaunay edge and excludes i itself. We reuse the same edges for the robot–object target T_t , making the robot reproduce the demonstrated neighborhood rather than form a new one. Anchor identities stay fixed across the sequence, while the tetrahedralization is recomputed each frame. The Laplacian coordinate of a node x_i is then defined as $\Delta_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} (x_i - x_j)$. It measures the offset from the neighbor centroid and is invariant to global translation.

Our IG extends OmniRetarget’s implementation in three ways. OmniRetarget represents the hand by one wrist node and cannot resolve finger-level contact. Its mesh spans one object plus terrain, whereas ours pools all scene objects into one graph. It scales human keypoints to the robot by a height ratio, whereas we retain true scale relative to the object.

Sequential per-frame optimization. We solve for robot wrist and joints that reproduce the human Laplacian coordinates under hard and soft constraints:

$$\begin{aligned} \min_{w_t, q_t} \quad & \mathcal{L}_{\text{IG}} + \mathcal{L}_{\text{coll}} + \mathcal{L}_{\text{temp}} \\ \text{s.t.} \quad & q_t \in \mathcal{Q}, \quad |q_t - q_{t-1}^*| \preceq \mathbf{v}_{\text{max}} dt. \end{aligned} \quad (1)$$

Superscript $\star$ denotes an optimizer solution. The interaction-graph term is the mean squared difference of Laplacian coordinates, $\mathcal{L}_{\text{IG}} = \frac{1}{N} \sum_{i=1}^N \|\Delta_i(\mathcal{S}_t) - \Delta_i(\mathcal{T}_t(w_t, q_t))\|_2^2$. Collisions are computed as a hinge penalty, $\mathcal{L}_{\text{coll}} = \sum_c \max(0, -g_c(w_t, q_t))^2$, where g_c is the distance of collision pair c minus its safety margin. Pairs cover robot–object and robot self-collision, excluding adjacent links. Let d_{rot} denote the geodesic rotation distance on $\text{SO}(3)$. The temporal regularizer $\mathcal{L}_{\text{temp}} = \|q_t - q_{t-1}^*\|_2^2 + \|p_t - p_{t-1}^*\|_2^2 + d_{\text{rot}}^2(R_{t-1}^*, R_t)$ keeps the joints and the wrist close to the previous frame solution. Here $\mathbf{v}_{\text{max}} \in \mathbb{R}^J$ gives per-joint velocity limits and $\preceq$ is elementwise. Wrist motion is only softly regularized by $\mathcal{L}_{\text{temp}}$. We solve the per-frame programs with sequential least-squares programming (SLSQP) [33]. Each program is warm-started from the previous solution, giving the seed retargeted references $\tau^* = \{(w_t^*, q_t^*)\}_{t=0}^{T-1}$.

B. Learned Object-Aware Retargeter

We amortize the optimizer into a feed-forward retargeter f_θ with autoregressive conditioning, trained on the hand–object interactions and seed retargeted references τ^* . The data spans many sequences and tasks, letting f_θ learn one shared retargeting function that generalizes to unseen data, instead of solving each frame in isolation. It predicts the robot configuration from the current observation and its own previous prediction, $f_\theta(K_t, P_t, \hat{q}_{t-1}, \hat{w}_{t-1}) = (\hat{w}_t, \hat{q}_t)$.

We represent each keypoint k_t^i in K_t by a 12D attribute, defined in the human wrist frame: position (3), its SMPL-X [32] joint orientation as a 6D rotation [34] (6), and linear velocity (3). A PointNet-style encoder [35] applies a shared pointwise multilayer perceptron (MLP) and global max pooling to the object-point features. We concatenate the descriptor with the flattened keypoint attributes as the per-frame feature ϕ_t . The joint and wrist MLP decoders read ϕ_t concatenated with their respective predictions from the previous frame, $\hat{q}_{t-1}$ or $\hat{w}_{t-1}$. This autoregressive conditioning mirrors the optimizer’s warm start and ablations (Table II) support this design. We keep it free-running rather than teacher-forced, so the network sees its own predictions during training and deployment, and condition the first frame on a zero-valued vector. The joint decoder passes its output through a per-joint sigmoid, $\hat{q}_t = q_{\text{low}} + \sigma(\cdot)(q_{\text{high}} - q_{\text{low}})$, keeping $\hat{q}_t \in \mathcal{Q}$ without a joint-limit loss. The wrist decoder outputs $\hat{p}_t \in \mathbb{R}^3$ and a continuous, easier-to-regress 6D rotation [34], converted to $\hat{R}_t \in \text{SO}(3)$.

1) *Supervised Training:* We first train f_θ autoregressively and fully supervised to reproduce the optimization outputs (w_t^*, q_t^*) , with the per-frame loss $\mathcal{L}_{\text{sup}} = \mathcal{L}_q + \mathcal{L}_w$. It equally weights the L_1 joint-angle loss $\mathcal{L}_q$ and the wrist loss $\mathcal{L}_w$, an L_1 translation term plus the geodesic $\text{SO}(3)$ rotation distance. We write its output as $\hat{\tau}^{\text{sup}} = \{(\hat{w}_t^{\text{sup}}, \hat{q}_t^{\text{sup}})\}_{t=0}^{T-1}$. With the seed reference as its only target, this step reproduces τ^* without improving it (Table I). Its role is to initialize the object-aware fine-tuning in Sec. III-B.2, so those losses

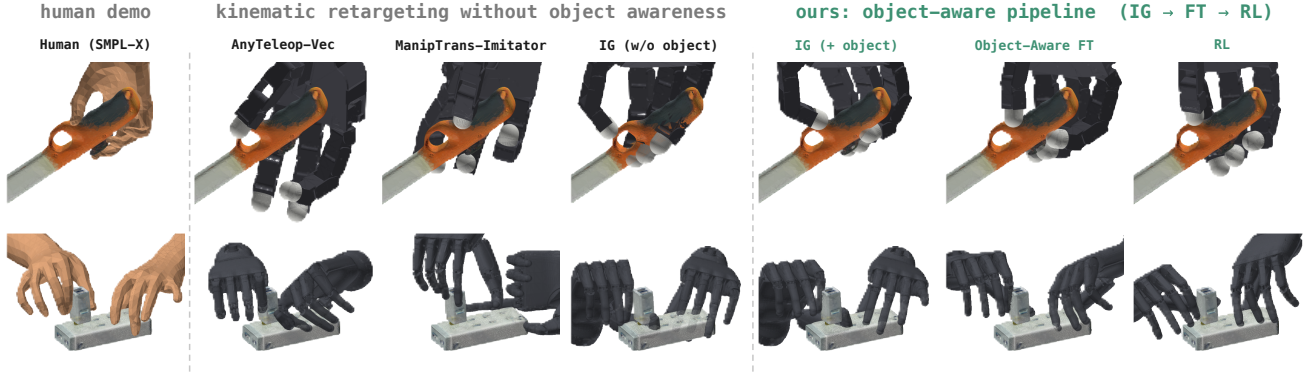


Fig. 3. **DEXOAK stages and downstream RL recover fingertip contacts missed by prior methods.** Two OakInk-v2 [6] task frames (top: ignite, Allegro hand [18]; bottom: unplug, Inspire hand [19]). Hand-only AnyTeleop-Vector [13], ManipTrans imitator [11], and *IG (w/o object)* optimizer leave fingertips off or inside the object. *IG (+ object)* optimizer, Object-Aware Fine-Tuning (FT), and RL (Sec. IV) progressively improve fingertip placement.

refine an existing solution rather than search for contacts from scratch. Without this initialization, fine-tuning is too weakly constrained to converge (ablations in Table II).

2) *Object-Aware Fine-Tuning (FT)*: The second step refines the same autoregressive network without labels. Matching Laplacian coordinates preserves the hand–object neighborhood but does not force the hand points onto the object surface, so contacts can remain inaccurate (Fig. 3, *IG (+ object)*). The objectives below act on these contacts directly. *Contact objectives.* For any point u , let $o(u)$ be the nearest point of the object cloud P_t and $d(u) = \|u - o(u)\|_2$ its distance to the object surface. Following Sec. III-A, hand point i has human position k_t^i and robot position r_t^i . We define three per-frame contact terms, illustrated in Fig. 2:

$$\mathcal{L}_{\text{surf}} = \sum_i \gamma_t^i d(r_t^i)^2, \quad (2)$$

$$\mathcal{L}_{\text{part}} = \sum_i \gamma_t^i \|r_t^i - o(k_t^i)\|_2^2, \quad (3)$$

$$\mathcal{L}_{\text{dir}} = \sum_i \gamma_t^i (1 - \cos \theta_t^i), \quad (4)$$

where θ_t^i is the angle between $r_t^i - o(k_t^i)$ and $k_t^i - o(k_t^i)$, and the gate γ_t^i activates a term only when the human point lies within a threshold ϵ of the object surface. Gating matters because contact terms are informative only near the object and otherwise add noise during approach and release. $\mathcal{L}_{\text{surf}}$ pulls each robot point onto the object surface. $\mathcal{L}_{\text{part}}$ pulls it to $o(k_t^i)$, the nearest sampled object point to the human contact, so each finger stays on the corresponding object part. $\mathcal{L}_{\text{dir}}$ makes the robot reach that point from the same side as the human, which separates opposite faces of thin geometry. *Regularization.* Three further terms keep the prediction feasible and stable where the contact terms give no signal. We reuse the collision penalty $\mathcal{L}_{\text{coll}}$ from Sec. III-A. In fact, $\mathcal{L}_{\text{surf}}$ is symmetric about the surface, so non-penetration is handled by $\mathcal{L}_{\text{coll}}$. $\mathcal{L}_{\text{temp}}$ uses the same form as in the optimizer and compares the consecutive network predictions $(\hat{w}_{t-1}, \hat{q}_{t-1})$ and $(\hat{w}_t, \hat{q}_t)$. The anchor term instead compares the prediction at frame t with the supervised network output at the same frame (Sec. III-B.1). It is $\mathcal{L}_{\text{anc}} = \|\hat{q}_t - \hat{q}_t^{\text{sup}}\|_2^2 + \|\hat{p}_t - \hat{p}_t^{\text{sup}}\|_2^2 + d_{\text{rot}}^2(\hat{R}_t^{\text{sup}}, \hat{R}_t)$.

The per-frame object-aware fine-tuning objective is the weighted sum: $\mathcal{L}_{\text{ft}} = \sum_{\ell} \lambda_{\ell} \mathcal{L}_{\ell}$, with $\ell \in \{\text{surf}, \text{part}, \text{dir}, \text{coll}, \text{anc}, \text{temp}\}$. The weights are tuned empirically and evaluated in Sec. V-C. The output is the refined

object-aware kinematic reference $\hat{\tau} = \{(\hat{w}_t, \hat{q}_t)\}_{t=0}^{T-1}$. Fig. 3, *Object-Aware FT*, illustrates this stage’s improvements.

IV. PHYSICS-BASED DYNAMIC RETARGETING

RL is well established for trajectory following in physics-based character and humanoid control [36, 37], and residual RL has since been used to correct base controllers for dexterous manipulation [11] and humanoid loco-manipulation [38]. We build on these methods for this stage.

For OakInk-v2 ablations (Table I), we use the residual RL of ManipTrans [11], and for ARCTIC generalization experiments (Table III), that of DexMachina [12]. Fixing RL within each table isolates the kinematic reference’s contribution. For the SOTA comparison across embodiments and datasets we use one unified residual RL, combining DeepMimic tracking [36] with the ManipTrans rewards [11] and the DexMachina object curriculum [12]. We find that residual RL needs the object curriculum on ARCTIC, so we keep it on both datasets and report a single protocol.

At step t , our unified residual policy observes simulator state s_t and one-step-ahead targets from the kinematic reference $\hat{\tau}$ and demonstrated object state (pose and velocity) x_t^o . It outputs a residual $\delta a_t = \pi(s_t, \hat{\tau}_{t+1}, x_{t+1}^o)$ that corrects a base action $\bar{a}_t$ derived from $\hat{\tau}_{t+1}$, so the policy only has to fix what reference following cannot achieve. The two are summed for the finger joints, $a_t = \bar{a}_t + \delta a_t$, and the actions are mapped onto the joint limits as proportional-derivative (PD) position targets. The wrist is driven by a proportional-integral-derivative (PID) controller that maps position and orientation errors to a force and torque, respectively, to which the residual is added. We train with reference-state initialization (RSI) [36] directly from $\hat{\tau}$ and the demonstrated object trajectory. Unlike prior work [10, 12], we do not pre-resolve its contacts in a simulator before dynamic retargeting. We adapt the domain-randomization ranges of the cited implementations (Sec. V-A), and otherwise follow the ManipTrans and DexMachina implementations.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

a) *Datasets:* We use two hand–object motion-capture datasets, training on each and evaluating on the subsets used

by SOTA methods. **OakInk-v2** [6]: bimanual and single-hand object manipulation. We train on the ManipTrans [11] set of 77 sequences across 42 objects (39 bimanual, 27 right-hand, 11 left-hand), reporting on the bimanual set for the SOTA comparison (Table IV) and the right-hand set for the ablations (Tables I and II). **ARCTIC** [5]: bimanual articulated-object manipulation. We train on 30 sequences (5 objects, 6 clips each) and report on the 7-clip subset used by DexMachina [12] and SPIDER [10] (Tables III and IV).

b) *Methods compared*: We compare two groups of methods from Sec. II. *Kinematic-reference* methods output a hand-centric, object-agnostic reference: optimization-based AnyTeleop [13] in its Position, Vector, and DexPilot [14] variants, and the learned GeoRT [25]. Since each produces only a reference, we run each through the same dynamic-retargeting backend as ManipTrans [11] to isolate reference quality. *Full retargeting pipelines* are ManipTrans, DexMachina [12], and SPIDER [10]. Only ManipTrans contributes a kinematic retargeter, its imitator policy, making it our primary comparison in Table I. As it reports no kinematic metrics, we score its imitator output ourselves.

c) *Robot hands*: **Allegro** [18] is a four-finger hand with 16 controllable DoF. **Inspire** [19] is a five-finger hand with 6 controllable DoF and 6 mimic joints.

d) *Kinematic metrics*: We score the kinematic reference with two metrics. **Tip-position error** (TPE, mm) is the mean Euclidean distance between the robot and human fingertips. **Contact completeness** (CC, %) asks whether the robot consistently replicates human contact. Human contact is defined by $d(k_t^i) < 10$ mm. Over frames with at least one such contact, CC is the fraction where every contacting robot fingertip lies within 10 mm of the surface. This deliberately conservative all-contact criterion requires the entire demonstrated contact pattern. Both metrics are diagnostic rather than complete predictors of downstream success, but they track it closely across our object-aware stages (Table I).

e) *Dynamic retargeting metrics*: Prior methods use different success definitions, so we report each under its native metric. **SR_{MT}** is the strict ManipTrans [11] criterion: a rollout succeeds when the object-rotation, object-translation, joint-position, and fingertip errors fall below 30° (0.524 rad), 3 cm, 8 cm, and 6 cm. **SR_{obj}** is the object-only SPIDER [10] criterion. It uses the same definitions as SR_{MT}, but scores object motion alone, with different bounds of 28.6° (0.5 rad) and 10 cm. **ADD-AUC** is DexMachina’s [12] object-tracking score, based on distances between object surface points under the achieved and demonstrated poses.

f) *Implementation*: We solve Eq. (1) using $M_o=64$ object anchors, with 5 mm robot–object and 8 mm self-collision margins for $\mathcal{L}_{\text{coll}}$. The retargeter uses $N_h=21$ keypoints per hand and an $N_p=1,024$ -point object cloud. Its PointNet-style encoder applies shared pointwise ReLU MLP layers of widths 64, 128, and 256, followed by global max pooling. Its two MLP decoders each have three 512-unit hidden layers. We train with AdamW and a cosine schedule for 600 supervised epochs at learning rate (lr) 10^{-3} and 24 object-aware fine-tuning epochs at lr 10^{-5} . The contact

TABLE I
KINEMATIC-REFERENCE COMPARISON ON THE ALLEGRO HAND
(RIGHT-HAND OAKINK-V2). ALL ROWS USE MANIPTRANS RESIDUAL
RL. RESULTS ARE OUR REPRODUCTIONS. BEST IN BOLD.

Kinematic reference	Kinematic metrics		RL metric
	Contact compl. ↑ (%)	TPE ↓ (mm)	SR _{MT} ↑ (%)
<i>Without Object (Hand-Only Reference)</i>			
AnyTeleop-Vector [13]	3.5	59.5	18.3
AnyTeleop-Position [13]	0.5	77.9	11.1
AnyTeleop-DexPilot [13, 14]	0.3	53.8	0.0
GeoRT [25]	1.0	58.6	0.8
ManipTrans [11]	5.8	49.2	20.6
<i>Object-Aware Reference (Ours)</i>			
IG optimizer	19.6	3.50	43.2
+ supervised training	19.2	3.50	43.6
+ Object-Aware FT (Full)	21.9	3.21	48.2

gate activates within $\epsilon=15$ mm of the surface and was varied from 5 to 25 mm with little effect on kinematic metrics. The fine-tuning weights ($\lambda_{\text{surf}}, \lambda_{\text{part}}, \lambda_{\text{dir}}, \lambda_{\text{coll}}, \lambda_{\text{anc}}, \lambda_{\text{temp}}$) are (5, 10, 1, 10, 1, 1). Ablations use uniform (1, 1, 1, 1, 1, 1), contact-heavy (50, 100, 20, 10, 1, 1), and regularization-heavy (5, 10, 1, 100, 50, 10) variants (Table II). Residual RL uses 4,096 environments and PPO [39], following [11, 12]. We scale object friction by [0.5, 3] and mass by $\pm 20\%$. Unless noted, results average 3 seeds in Tables I-III and 5 in Table IV, with each policy evaluated over 256 rollouts. Baseline results in Tables III and IV are from the cited papers. All experiments use an NVIDIA H100.

B. Ablations of Object Awareness

We compare object-aware and hand-only references on the OakInk-v2 Allegro right-hand tasks, all under the same ManipTrans residual RL (Table I). Hand-only references reach at most 20.6% RL success, with near-zero contact completeness and centimeter-scale tip errors. Object-aware references substantially improve both kinematic metrics and reach 48.2% success, 2.3× the ManipTrans result. Placing fingers at demonstrated contacts lets residual RL refine dynamics rather than discover contacts. Fig. 3 illustrates the improvement from *IG (w/o object)* to *IG (+ object)*.

The object-aware rows in Table I isolate our method’s stages. The IG Laplacian optimizer supplies the reference, supervised training reproduces it, and object-aware fine-tuning improves every metric. DEXOAK adds 5.0 percentage points over the optimizer. The same ordering holds on ARCTIC (Table III: IG optimizer vs. DEXOAK). This supports our second contribution: the learned retargeter is differentiable, so we improve it under dataset-wide contact objectives rather than merely amortize the solver.

C. Our Method Ablations

Table II ablates our method’s components on Allegro right-hand OakInk-v2 tasks (kinematic metrics only). Removing any component degrades the kinematic metrics, justifying the design choices of Sec. III. Dropping the object anchors leaves a hand-only optimizer and yields 2.8% contact completeness and 41.5 mm TPE, showing that the object

TABLE II

COMPONENT ABLATIONS OF DEXOAK ON THE ALLEGRO HAND (RIGHT-HAND OAKINK-v2). KINEMATIC METRICS ONLY, WITHOUT RL. EACH ROW CHANGES ONE DESIGN CHOICE FROM THE FULL MODEL, WITH ALL ELSE FIXED.

Variant	Contact completeness $\uparrow$ (%)	TPE $\downarrow$ (mm)
<i>Interaction-Graph Laplacian Optimizer</i>		
without object anchors	2.8	41.5
without wrist node	19.6	3.51
<i>Architecture and Training</i>		
without object cloud	20.0	3.83
without autoregression	16.1	9.29
single-sequence training	20.9	3.53
without supervised training	10.3	36.8
<i>Object-Aware Losses (Leave-One-Out)</i>		
without surface-contact	13.2	6.84
without object-part	19.2	7.71
without contact-direction	18.8	7.25
<i>Loss-Weight Configurations</i>		
uniform	19.2	5.03
contact-heavy	20.8	4.11
regularization-heavy	19.3	3.85
Full model	21.9	3.21

nodes carry the demonstrated contact geometry. Skipping the supervised stage is almost as damaging (10.3% and 36.8 mm): without amortizing the optimizer first, the model has no usable initialization and fails to learn a useful re-targeting from scratch. Autoregression, the learned-network counterpart of the optimizer’s warm start, is also important, helping smooth predictions. Removing the object cloud from the network input costs little (20.0% and 3.83 mm), as object information still enters through the losses. All tested weightings remain close to the full model and substantially outperform the hand-only references from Table I. The benefit thus comes from the loss terms themselves rather than their precise balance.

D. Generalization

We test whether our retargeter transfers across datasets without re-optimization. We evaluate on ARCTIC with the Allegro hand under DexMachina’s RL (Table III). We report speed and ADD-AUC. Speed is calculated as evaluated frames divided by total target-data kinematic-stage wall-clock time, including inference, per-frame optimization, and any one-time training or fine-tuning, excluding RL. The top block trains and evaluates DEXOAK on ARCTIC, following our typical pipeline and providing baselines for the generalization rows below. In the bottom block, our OakInk-v2 DEXOAK model is applied zero-shot to the ARCTIC data, or after object-aware fine-tuning on ARCTIC (+ *Object-Aware FT*). Neither transfer setting requires per-frame optimization.

Zero-shot, the OakInk-v2 model reaches 87.9 ADD-AUC, above DexMachina at 83.4, and generates references 7 times faster than our in-domain optimizer (108.9 vs. 15.5 Hz). Object-aware fine-tuning on ARCTIC raises ADD-AUC to 90.0, near in-domain DEXOAK at 91.3. Thus, our trained

TABLE III

CROSS-DATASET TRANSFER OF DEXOAK ON THE ALLEGRO HAND (BIMANUAL OAKINK-v2 $\rightarrow$ ARCTIC). ALL ROWS USE DEXMACHINA RL. SPEED IS EVALUATED FRAMES DIVIDED BY TOTAL TARGET-DATA KINEMATIC-STAGE TIME, INCLUDING ANY OPTIMIZATION, TRAINING, OR FINE-TUNING BUT EXCLUDING RL. ADD-AUC MEASURES DYNAMIC SUCCESS. BEST IN **BOLD**, SECOND-BEST UNDERLINED.

Method	Speed (Hz) $\uparrow$	ADD-AUC (%) $\uparrow$
<i>In-domain on ARCTIC</i>		
DexMachina [12]	13.4	83.4
Ours: IG optimizer	15.5	88.9
Ours: DEXOAK	1.4	91.3
<i>Ours: generalization OakInk-v2 $\rightarrow$ ARCTIC</i>		
Zero-shot	108.9	87.9
+ Object-Aware FT on ARCTIC	<u>57.2</u>	<u>90.0</u>

TABLE IV

DEXOAK+RL VS. SOTA RETARGETING. METHODS USE THEIR OWN DYNAMIC REFINEMENT, WHILE OURS USES THE RESIDUAL RL OF SEC. IV. –: NOT REPORTED. * SPIDER USES MORE OAKINK-v2 SEQUENCES, MAKING ITS COMPARISON INDICATIVE. BEST IN **BOLD**.

Method	SR _{MT} (%) $\uparrow$	SR _{obj} (%) $\uparrow$	ADD-AUC (%) $\uparrow$
<i>OakInk-v2 (bimanual), Inspire hand</i>			
ManipTrans [11]	39.5	–	–
SPIDER [10]	–	47.9*	–
DEXOAK+RL	48.4	58.7	55.1
<i>OakInk-v2 (bimanual), Allegro hand</i>			
SPIDER [10]	–	45.9*	–
DEXOAK+RL	58.6	64.1	61.5
<i>ARCTIC, Inspire hand</i>			
DexMachina [12]	–	67.1	62.7
SPIDER [10]	–	42.0	–
DEXOAK+RL	58.3	83.4	77.9
<i>ARCTIC, Allegro hand</i>			
DexMachina [12]	–	–	83.4
DEXOAK+RL	67.2	97.3	92.1

retargeter generalizes zero-shot to new datasets, scaling DEXOAK to data it was never optimized on. Moreover, target-data object-aware fine-tuning improves ADD-AUC while remaining far faster than the in-domain DEXOAK (+ *Object-Aware FT* vs. *Ours*: DEXOAK in Table III).

E. Comparison to State of the Art

Table IV compares our DEXOAK+RL pipeline to ManipTrans, DexMachina, and SPIDER on both hands over bimanual OakInk-v2 and ARCTIC, each under its own dynamic retargeting and native metric. *Ours* uses the residual RL of Sec. IV, so the ARCTIC numbers here are not directly comparable to Table III, which runs DexMachina’s RL. Each baseline runs its published pipeline, so this table compares complete systems rather than isolating the kinematic reference, which Tables I and III establish under a fixed backend. On ARCTIC with the Inspire hand, DEXOAK+RL doubles SPIDER’s object success, 83.4 against 42.0 SR_{obj}, and adds more than 15 ADD-AUC points to DexMachina to reach 77.9. The ordering is the same on OakInk-v2, where DEXOAK+RL outperforms ManipTrans under SR_{MT} and



Fig. 4. **Qualitative results across embodiments and datasets.** Kinematic references (left) and dynamically refined trajectories (right). The top three rows compare ManipTrans [11], SPIDER [10], and DEXOAK(+RL) on *uncap alcohol lamp* (OakInk-v2 [6], Inspire [19]). The remaining rows show DEXOAK(+RL) on *use box* (ARCTIC [5], Allegro [18]) and a failure on *shake lab container* (OakInk-v2, Allegro), where the grasp breaks during shaking.

SPIDER under SR_{obj} . Across both hands, DEXOAK+RL outperforms each reported SOTA method under its native metric. Interestingly, RL success is higher with Allegro than Inspire (67.2 vs. 58.3 SR_{MT} on ARCTIC), despite its bulkier fingers. This may reflect Allegro’s 16 controllable DoF versus Inspire’s 6, allowing Allegro to cover a wider range of human hand configurations.

Qualitative results (Fig. 4) mirror Table IV. On the OakInk-v2 task *uncap alcohol lamp*, dynamic retargeting fails for ManipTrans and SPIDER, while DEXOAK+RL executes the full motion. Nevertheless, failures remain. On highly dynamic tasks (e.g., *shake lab container*), the grasp breaks mid-trajectory because kinematic retargeting lacks knowledge of sequence dynamics and the residual policy alone must supply momentum control. Further examples are in the supplementary video.

F. Real-Robot Experiments

These experiments evaluate the DEXOAK+RL pipeline for generating executable robot trajectories on 5 OakInk-v2 [6] tasks. For each task–method pair, we evaluate 3 trained policies (one per seed). From each policy, we retain the first rollout satisfying SR_{MT} in a fixed evaluation order, record its wrist poses and finger joint angles, and replay it open-loop 4 times on a UR5 + right Allegro setup, yielding up to 12 trials per task (fewer when some policies produce no successful rollout). We replay the 60 Hz rollout in joint space, obtaining UR5 targets by IK from the recorded wrist poses and sending joint-position targets to both the UR5 and Allegro. Before each replay, the object is manually reset to its marked initial

pose, within 5 mm of the simulated initialization. Object dimensions match their mocap counterparts within 2 mm. A replay succeeds if it completes the demonstrated object motion: object transport to the demonstrated final position, gate/laptop closure, bottle tilt-and-return, plug removal, knife cutting-stroke-and-return, or button press. In Table V, brackets report seed-averaged simulation SR_{MT} (how often the policy succeeds in simulation), while x/y reports hardware repeatability of the selected rollouts. Hardware success is therefore conditional on simulation success. We compare with ManipTrans [11], trained with its original recipe.

DEXOAK+RL achieves near-perfect real-robot performance on the first three tasks: *Take out*, *Close gate* and *Pour*. On *Unplug* and *Cut*, abrupt or high-frequency motions can cause the plug or knife to slip from the grasp, exacerbated by open-loop sensitivity to initialization and consistent with the dynamic failures observed in simulation (Fig. 4). Low simulation success further limits replayable *Cut* trajectories. Compared with ManipTrans, DEXOAK+RL improves SR_{MT} on all 5 tasks, and matches or substantially exceeds its available hardware success, with the clearest gain on *Unplug*.

We additionally apply the frozen DEXOAK retargeter zero-shot to two tasks held out from all retargeter training: *Close Laptop* and *Press Button*, followed by the same task-specific RL stage. Both are non-prehensile interactions with objects from unseen categories. They achieve 99% and 100% simulation success, respectively, and 10/12 hardware successes each. These results show that DEXOAK generalizes to unseen interactions, producing references that RL can

TABLE V

REAL-ROBOT RESULTS. THE FIRST SUCCESSFUL SIMULATION ROLLOUT PER POLICY (3 SEEDS) IS REPLAYED 4 TIMES ON A UR5 + RIGHT ALLEGRO. ENTRIES SHOW HARDWARE SUCCESSES/TRIALS [SIMULATION SR_{MT}]. DENOMINATORS <12: FEWER THAN 3 POLICIES YIELDED A SUCCESSFUL ROLLOUT. -: NO SUCCESSFUL ROLLOUT.

Method	Take out	Close gate	Pour	Unplug	Cut
DEXOAK+RL	12/12 [90%]	12/12 [41%]	10/12 [95%]	8/12 [93%]	4/8 [3%]
ManipTrans [11]	6/12 [83%]	12/12 [12%]	8/12 [16%]	0/4 [1%]	- [0%]

refine into replayable hardware trajectories. Representative examples are shown in Fig. 1 and the supplementary video.

VI. CONCLUSION

We introduce DEXOAK, an object-aware kinematic framework that retargets human hand-object demonstrations into dexterous robot trajectories. It preserves object-relative interaction geometry through interaction-graph Laplacian optimization, amortizes it with a feed-forward retargeter refined by object-aware contact losses, and uses its output as the base action for residual RL. Across single-hand and bimanual tasks on two robot embodiments, DEXOAK+RL clearly outperforms SOTA retargeting methods. Its RL-refined trajectories also replay successfully on real hardware.

Future work could share dynamics corrections across tasks and hands through one task- and embodiment-conditioned residual policy, and reach beyond motion-capture by retargeting hand-object trajectories reconstructed from RGB video. Both steps would extend DEXOAK into an engine that turns human observation into executable dexterous robot data.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, “OpenVLA: An open-source vision-language-action model,” in *CoRL*, 2024.
- [2] Physical Intelligence, “ $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities,” *arXiv*, 2026.
- [3] C. Wang, H. Shi, W. Wang, R. Zhang, L. Fei-Fei, and C. K. Liu, “DexCap: Scalable and portable mocap data collection system for dexterous manipulation,” in *RSS*, 2024.
- [4] Y.-W. Chao, W. Yang, Y. Xiang, P. Molchanov, A. Handa, J. Tremblay, Y. S. Narang *et al.*, “DexYCB: A benchmark for capturing hand grasping of objects,” in *CVPR*, 2021.
- [5] Z. Fan, O. Taheri, D. Tzionas, M. Kocabas, M. Kaufmann, M. J. Black, and O. Hilliges, “ARCTIC: A dataset for dexterous bimanual hand-object manipulation,” in *CVPR*, 2023.
- [6] X. Zhan, L. Yang, Y. Zhao, K. Mao, H. Xu, Z. Lin, K. Li, and C. Lu, “OAKINK2: A dataset of bimanual hands-object manipulation in complex task completion,” in *CVPR*, 2024.
- [7] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, “R3M: A universal visual representation for robot manipulation,” in *CoRL*, 2023.
- [8] R. Yang, Q. Yu, Y. Wu, R. Yan, B. Li, A.-C. Cheng, X. Zou, Y. Fang *et al.*, “EgoVLA: Learning vision-language-action models from egocentric human videos,” *arXiv*, 2025.
- [9] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar, “MimicPlay: Long-horizon imitation learning by watching human play,” in *CoRL*, 2023.
- [10] C. Pan, C. Wang, H. Qi, Z. Liu, H. Bharadhwaj, A. Sharma, T. Wu, G. Shi, J. Malik, and F. R. Hogan, “SPIDER: Scalable physics-informed dexterous retargeting,” in *IROS*, 2026.
- [11] K. Li, P. Li, T. Liu, Y. Li, and S. Huang, “ManipTrans: Efficient dexterous bimanual manipulation transfer via residual learning,” in *CVPR*, 2025.
- [12] M. Zhao, Y. Hou, D. Fox, Y. Narang, A. Mandlekar, and S. Song, “DexMachina: Functional retargeting for bimanual dexterous manipulation,” in *ICML*, 2026.
- [13] Y. Qin, W. Yang, B. Huang, K. Van Wyk, H. Su, X. Wang, Y.-W. Chao, and D. Fox, “AnyTeleop: A general vision-based dexterous robot arm-hand teleoperation system,” in *RSS*, 2023.
- [14] A. Handa, K. Van Wyk, W. Yang, J. Liang, Y.-W. Chao, Q. Wan, S. Birchfield, N. D. Ratliff, and D. Fox, “DexPilot: Vision-based teleoperation of dexterous robotic hand-arm system,” in *ICRA*, 2020.
- [15] L. Yang, X. Huang, Z. Wu, A. Kanazawa, P. Abbeel, C. Sferazza, C. K. Liu, R. Duan, and G. Shi, “OmniRetarget: Interaction-preserving data generation for humanoid whole-body loco-manipulation and scene interaction,” in *ICRA*, 2026.
- [16] J. Nocedal and S. J. Wright, *Numerical optimization*. Springer, 2006.
- [17] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. Aparicio Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *ICRA*, 2019.
- [18] Wonik Robotics, “Allegro hand,” <https://www.wonikrobotics.com/research-robot-hand>, accessed: 2026-06-26.
- [19] Inspire-Robots, “Dexterous hand (RH56 series),” <https://inspire-robots.store/>, accessed: 2026-06-26.
- [20] R. Punamiya, D. Patel, P. Aphiwetsa, P. Kuppili, L. Y. Zhu, S. Kareer, J. Hoffman, and D. Xu, “EgoBridge: Domain adaptation for generalizable imitation from egocentric human data,” in *NeurIPS*, 2025.
- [21] H. G. Singh, A. Loquercio, C. Sferazza, J. Wu, H. Qi, P. Abbeel, and J. Malik, “Hand-object interaction pretraining from videos,” in *ICRA*, 2025.
- [22] H. Chen, T. Dong, T. Wu, L. Wang, Y. Jangir, Y. Niu *et al.*, “Dexterous manipulation policies from RGB human videos via 4D hand-object trajectory reconstruction,” *arXiv*, 2026.
- [23] X. Liu, J. Adalibekie, Q. Han, Y. Qin, and L. Yi, “DexTrack: Towards generalizable neural tracking control for dexterous manipulation from human references,” in *ICLR*, 2025.
- [24] B. Zhou, H. Yuan, Y. Fu, and Z. Lu, “Learning diverse bimanual dexterous manipulation skills from human demonstrations,” in *AAAI*, 2026.
- [25] Z.-H. Yin, C. Wang, L. Pineda, K. Bodduluri, T. Wu, P. Abbeel, and M. Mukadam, “Geometric retargeting: A principled, ultrafast neural hand retargeting algorithm,” in *IROS*, 2025.
- [26] J. Wu, S. Yao, G. He, X. Liu, Z. Zeng, X. Jiang, H. Yang, W. Zhang, and H. Zhao, “TopoRetarget: Interaction-preserving retargeting for dexterous manipulation,” *arXiv*, 2026.
- [27] J. Ren, Z. Jiang, Y. Yang, H. Jia, H. Jiang *et al.*, “C2Dex: Contact-consistent reconstruction and retargeting for dexterous manipulation from monocular video,” *arXiv*, 2026.
- [28] Y. Feng, N. Leung, J. Wang, L. Yang, H. Qi, and P. Culbertson, “A minimalist retargeting-guided reinforcement learning recipe for dexterous manipulation,” *arXiv*, 2026.
- [29] P. Grady, C. Tang, C. D. Twigg, M. Vo, S. Brahmabhatt, and C. C. Kemp, “ContactOpt: Optimizing contact to improve grasps,” in *CVPR*, 2021.
- [30] D. Turpin, L. Wang, E. Heiden, Y.-C. Chen, M. Macklin, S. Tsogkas, S. J. Dickinson, and A. Garg, “GraspD: Differentiable contact-rich grasp synthesis for multi-fingered hands,” in *ECCV*, 2022.
- [31] H. Zhang, S. Christen, Z. Fan, L. Zheng, J. Hwangbo, J. Song, and O. Hilliges, “ArtiGrasp: Physically plausible synthesis of bi-manual dexterous grasping and articulation,” in *3DV*, 2024.
- [32] G. Pavlakos, V. Choutas, N. Ghorbani, T. Bolkart, A. A. Osman, D. Tzionas, and M. J. Black, “Expressive body capture: 3D hands, face, and body from a single image,” in *CVPR*, 2019.
- [33] A. J. Joshy and J. T. Hwang, “PySLSQP: A transparent python package for the SLSQP optimization algorithm modernized with utilities for visualization and post-processing,” *Journal of Open Source Software*, 2024. [Online]. Available: <https://doi.org/10.21105/joss.07246>
- [34] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, “On the continuity of rotation representations in neural networks,” in *CVPR*, 2019.
- [35] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “PointNet: Deep learning on point sets for 3D classification and segmentation,” in *CVPR*, 2017.
- [36] X. B. Peng, P. Abbeel, S. Levine, and M. van de Panne, “DeepMimic: Example-guided deep reinforcement learning of physics-based character skills,” *TOG*, 2018.
- [37] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu, “BeyondMimic: From motion tracking to versatile humanoid control via guided diffusion,” *Science Robotics*, 2026.
- [38] S. Zhao, Y. Ze, Y. Wang, C. K. Liu, P. Abbeel, G. Shi, and R. Duan, “ResMimic: From general motion tracking to humanoid whole-body loco-manipulation via residual learning,” *arXiv*, 2025.
- [39] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv*, 2017.